

Modern Database Workflows: Isolated PostgreSQL Containers and Remote Access Solutions

Containers, GUI Tools, and Secure Tunneling

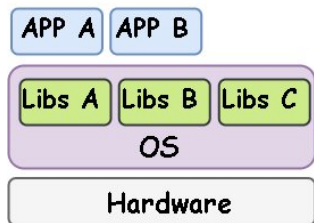
Bidur Devkota, PhD

July 9, 2025

From Monoliths to Containerized Databases

Traditional Approach:

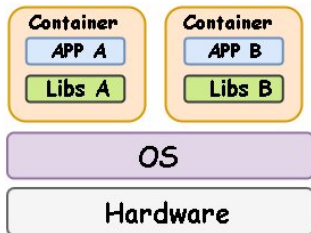
- Databases installed on bare metal
- Manual configuration
- Dependency conflicts
- Hard to replicate



Traditional OS

Modern Approach:

- Containerized isolation
- Portable environments
- Reproducible setups
- DevOps friendly

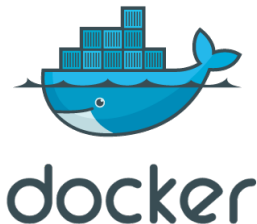


Containers

<https://t.ly/H0pdv>

PostgreSQL in Containers: Key Benefits

- **Consistency:** Identical environments everywhere
- **Security:** Isolated processes reduce attack surface
- **Scalability:** Spin up/down instances easily
- **DevOps Friendly:** CI/CD pipeline integration



<https://t.ly/-MJzd>

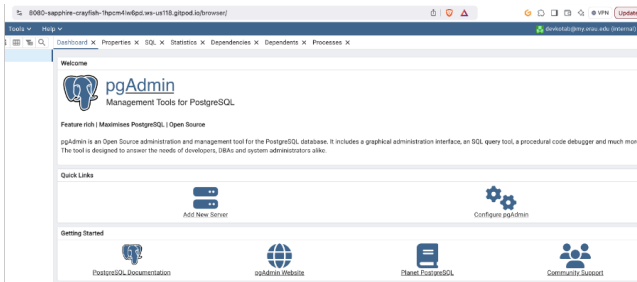
GUI vs. CLI: Why Use pgAdmin?

For Developers

- Visual query builder
- Schema visualization
- Import/export tools

For Admins

- User/role management
- Dashboard monitoring
- Maintenance tools



Why Can't We Expose PostgreSQL Directly?

Problems

- Cloud environments block direct access
- Firewall restrictions
- Dynamic IP addresses
- Security policies

Solution: Chisel Tunneling

- Secure encrypted bridge
- No open ports required
- Works behind NAT

Where This Workflow Shines

Collaborative Analytics

- Data scientists query from Colab
- Shared datasets
- Real-time collaboration

Remote Development

- Global team access
- Consistent environments
- No local setup needed

Education

- Pre-configured DBs for students
- No installation headaches
- Focus on learning SQL

1. **PostgreSQL:** An open-source relational database system (RDBMS) known for:
 - ACID compliance, scalability, and extensibility.
 - Supports SQL queries.
2. **pgAdmin:** A web-based GUI for PostgreSQL that provides:
 - Database management (tables, queries, users).
 - Visual tools for backups, imports, and monitoring.

3. Chisel Tunneling: A TCP/UDP tunnel tool to securely expose local services (e.g., PostgreSQL) to remote clients (e.g., Google Colab). **Why Needed?**

- GitPod/containers run in isolated environments (no direct remote access).
- Chisel bypasses firewall/NAT restrictions securely.
- Enables real-time collaboration (e.g., Colab accessing GitPod's PostgreSQL).

Task 1: Use Docker to containerize PostgreSQL and pgAdmin for database management.

Key Steps:

- Pull and run PostgreSQL container.
- Create databases/tables via CLI.
- Deploy pgAdmin for GUI management.

Task 2: Tunneling to enable real-time collaboration among Colab and GitPod apps.

Key Steps:

- Pull and install Chisel.
- Tunnel access via Chisel (Google Colab integration).

Step 1: Run PostgreSQL Container

```
# Pull PostgreSQL image
docker pull postgres

# Run container with volume mapping
docker run --name postgres_container \
  -e POSTGRES_PASSWORD=12345 \
  -d -p 5432:5432 \
  -v /workspace/empty/pg_files:/var/lib/postgresql/data \
  postgres
```

Verify:

```
docker exec -it postgres_container psql -U postgres
CREATE DATABASE flight;
\l # List databases
```

Step 2: Create Tables

```
CREATE TABLE faa_airport_stats (  
    id INTEGER PRIMARY KEY,  
    passenger_count INTEGER,  
    Quarter TEXT,  
    Avg_Comp_Airfare NUMERIC(6,2),  
    GDP NUMERIC(12,2),  
    -- Additional columns...  
);
```

Import Data:

```
psql -U postgres -d flight -c "\copy faa_airport_stats  
FROM 'flights-export.csv' WITH (FORMAT csv, HEADER) "
```

Step 3: Deploy pgAdmin

```
# Pull and run pgAdmin
docker pull dpape/pgadmin4
docker run --name pgadmin_container \
  -p 8080:80 \
  -e PGADMIN_DEFAULT_EMAIL=im.bidur@gmail.com \
  -e PGADMIN_DEFAULT_PASSWORD=12345 \
  -d dpape/pgadmin4
```

Access: <https://8080-<gitpod-url>.gitpod.io> **Connect to PostgreSQL:** Host: 172.17.0.2 (container IP), Port: 5432, DB: postgres.

Step 4: Tunnel with Chisel

GitPod (Server):

```
curl https://i.jpillora.com/chisel! | bash
chisel server --port 5433
```

Google Colab (Client):

```
nohup chisel client https://5433-gitpod-url.gitpod.io \
0.0.0.0:5438:5432 &
```

Use in Colab:

```
import psycopg2
conn = psycopg2.connect(
    host="0.0.0.0", port=5438,
    user="postgres", password="12345", dbname="flight"
)
```

Activities Recap:

- 1 Containerized PostgreSQL + pgAdmin using Docker.
- 2 Created/imported tables via CLI and GUI.
- 3 Securely accessed DB from Colab via Chisel tunnel.

Benefits:

- Isolation via containers.
- GUI (pgAdmin) + CLI flexibility.
- Remote access for collaboration.